



Agentic Bug Triage & Routing System

User Manual

Complete guide from first login to AI analysis results and custom connectors.

1. Introduction	4
1.1 What This System Does	4
1.2 Severity Levels at a Glance	4
2. Getting Started	5
2.1 What You Need Before You Begin	5
2.2 Opening the Application	5
3. Logging In	6
3.1 How to Log In	6
3.2 Demo Credentials	6
3.3 Logging Out	7
4. Navigation	8
4.1 Navigation Bar Elements	8
5. Dashboard	9
5.1 Live Stats Strip	9
5.2 Stat Cards	9
5.3 Connected Systems Panel	9
5.4 Recent Triage Activity	10
6. Bug List	11
6.1 Mini Stats Strip	11
6.2 Filtering and Searching	11
6.3 Direct Triage Bar	12
6.4 Understanding Bug Rows	12
6.5 Triage State Indicators	12
6.6 Pagination and Refresh	13
7. Starting a Triage	14
Option A - From the Bug List	14
Option B - By Bug ID	14
8. Triage Page (Live Analysis)	15
8.1 The Loading Screen	15
8.2 The Results Grid	16
8.3 Panel 01 - Bug Context	17
8.4 Panel 02 - Related Issues	17
8.5 Panel 03 - Knowledge Base	17
8.6 Panel 04 - AI Summary	17

9. History Page.....	19
9.1 Reading the History Table	19
9.2 Actions You Can Take.....	20
10. Settings Page	21
10.1 Connection Status Badges	21
10.2 Adding a New Connection.....	21
10.3 Managing Existing Connections	22
11. Adding a Custom Connector	23
11.1 Overview.....	23
11.2 The Five Steps	23
11.3 Quick Reference - Connector Checklist	25
12. Common Tasks - Quick Reference	26
13. Troubleshooting	27
13.1 I Cannot Log In	27
13.2 The Bug List Is Empty	27
13.3 The Triage Pipeline Is Stuck on "Running..."	27
13.4 The Page Shows "Triage Expired"	27
13.5 AI Summary Shows "Fallback Analysis Used"	27
13.6 A Connection Shows "Error" or "Timeout"	27
13.7 My Custom Connector Type Does Not Appear in the Settings Dropdown	28
14. Glossary.....	29

1. Introduction

Welcome to the Agentic Bug Triage & Routing System. This manual guides you through everything you need as an Engineer - from logging in for the first time to triaging bugs, reading AI-generated analysis results, and writing your own custom connectors.

The system connects to multiple bug-tracking platforms (GitHub, Jira, and Bugzilla) and automatically collects, groups, and prioritises bug reports into one unified dashboard. A four-step AI pipeline then analyses each bug and produces a severity rating, confidence score, root cause, and recommended actions.

1.1 What This System Does

- Pulls bugs from GitHub, Jira, and Bugzilla into one unified list
- Groups related bugs from different systems automatically
- Runs a 4-step AI pipeline to analyse any bug you choose
- Produces a severity rating (P0-P3), confidence score, root cause, and recommended actions
- Shows relevant knowledge base articles for each bug
- Keeps a full history of every analysis run
- Allows Engineers to write and register custom connectors for new platforms

1.2 Severity Levels at a Glance

The system uses four severity levels assigned by the AI:

Level	Name	Meaning	Expected Response
P0	Critical	Production is down or data loss is occurring.	Immediate action required
P1	High	A major feature is broken with no workaround.	Fix within the day
P2	Medium	A significant issue; a workaround is available.	Fix this sprint
P3	Low	A minor or cosmetic issue.	Schedule for a future sprint

2. Getting Started

2.1 What You Need Before You Begin

- The project services are running on your machine or your team's server
- You have been given a login email and password
- You are using a modern web browser (Chrome, Firefox, Edge, or Safari)

NOTE: If you do not have login credentials, contact your project lead. Demo credentials are displayed on the login screen if enabled.

2.2 Opening the Application

Open your web browser and go to the address your team has configured. For a local installation, the default URL is:

```
http://localhost:5173
```

If deployed to a shared server, use the URL provided by your project lead.

3. Logging In

When you open the application, you are taken to the Login page. This is the starting point for all Engineers.

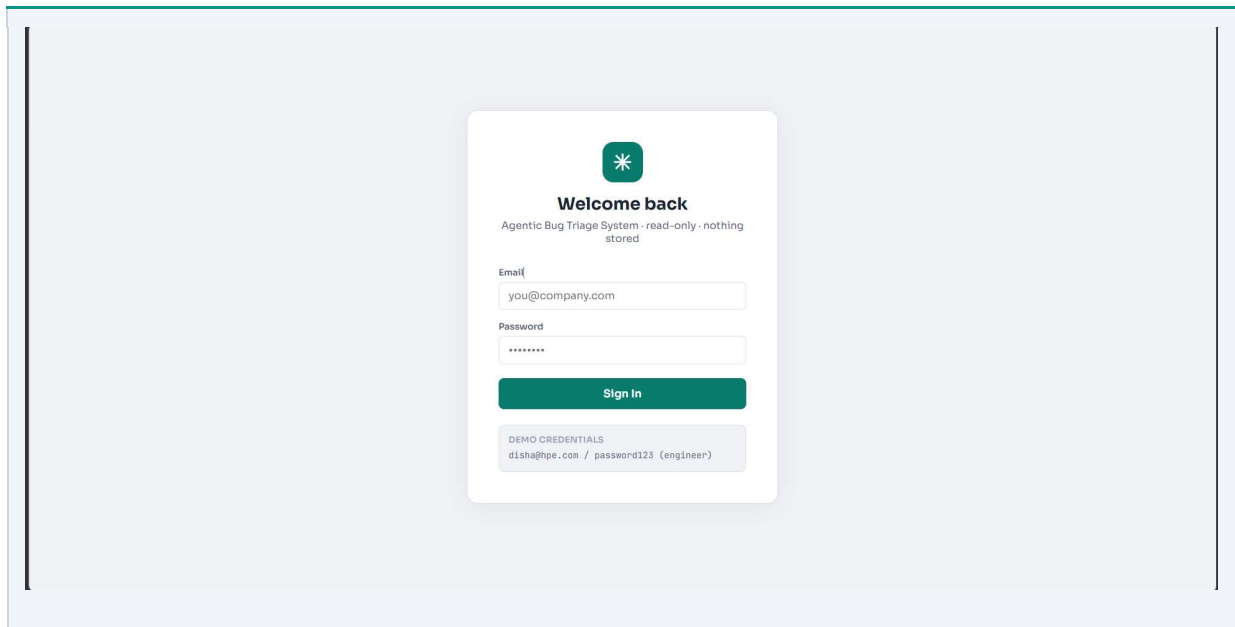


Figure 1 - Login page: email field, password field, Sign In button, and demo credentials

3.1 How to Log In

1. Type your email address in the Email field
2. Type your password in the Password field
3. Click the Sign In button
4. If your credentials are correct, you will be taken to the Bug List page

IMPORTANT: If you see an error, double-check your email and password for typos or extra spaces. If the problem continues, ask your project lead to reset your password.

3.2 Demo Credentials

If demo mode is enabled, the login screen shows sample credentials at the bottom of the card. These are useful for exploring the application without a real account.

IMPORTANT: Never use demo credentials in a production environment. Change them immediately after initial setup.

3.3 Logging Out

To log out, click the Logout button in the top-right corner of the navigation bar. You will be returned to the Login page and your session will end immediately.

4. Navigation

Once logged in, the navigation bar appears at the top of every page. It gives you access to all parts of the application and shows important status information.

4.1 Navigation Bar Elements

Element	What It Does
Logo (asterisk icon)	Click to go to the Bug List from anywhere in the app.
Dashboard	Opens the System Overview page with live statistics and recent activity.
Bugs	Opens the unified Bug List - the main workspace.
History	Opens the Triage History log of all completed AI analysis sessions.
Settings	Opens the Connections and configuration area.
Systems chip (e.g. "2 systems connected")	Shows how many source connectors are currently active.
Your email address	Confirms which account is currently active.
Role badge (Engineer)	Confirms your access level.
Logout button	Ends your session and returns you to the Login page.

5. Dashboard

The Dashboard gives you an instant health check of all bugs and systems. It is the best place to start your working day before diving into the Bug List. Click Dashboard in the navigation bar to open it.

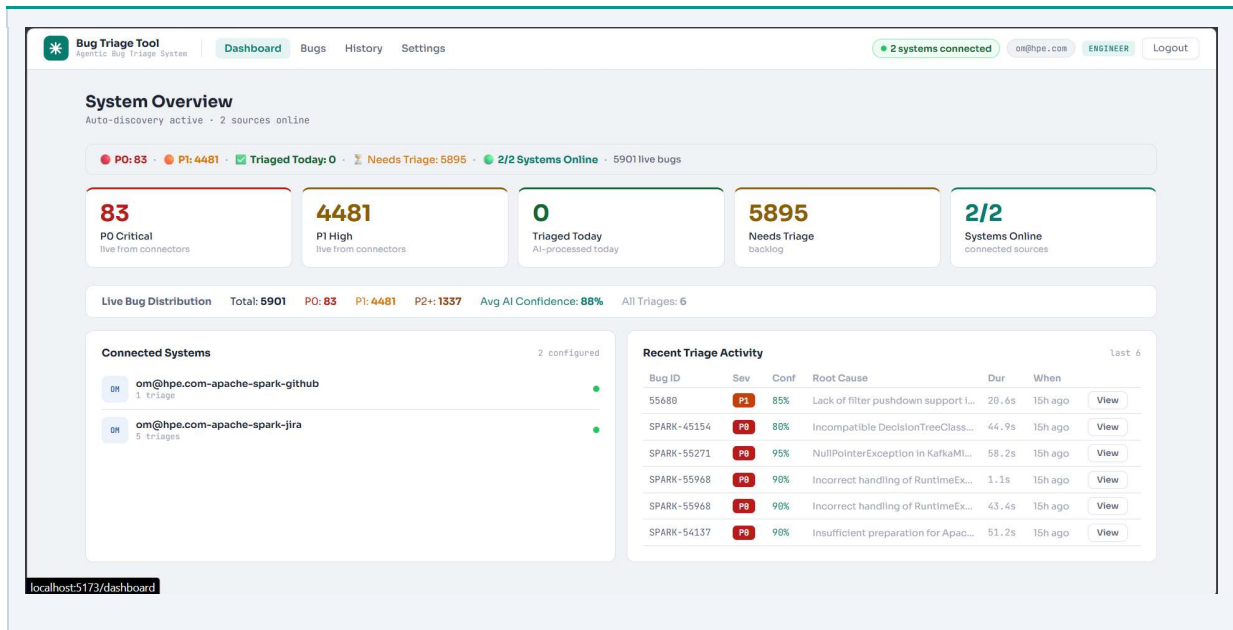


Figure 2 - System Overview Dashboard: stats strip, five stat cards, connected systems panel, and recent triage activity

5.1 Live Stats Strip

The bar at the top shows live counts from all connected sources:

- P0 - number of critical bugs currently open across all systems
- P1 - number of high-priority bugs currently open
- Triaged Today - how many bugs the AI has analysed today
- Needs Triage - bugs not yet analysed or whose analysis has expired
- Systems Online - how many connectors are active (e.g. "2/2 Systems Online")

5.2 Stat Cards

Below the stats strip are five cards giving more detail on each live count. Each card shows the count, a label, and a note about where the number comes from (e.g. "live from connectors" or "AI-processed today").

5.3 Connected Systems Panel

The left panel lists every connected source system with how many triage sessions have been completed for bugs from that source. A green dot indicates the system is online and responding.

5.4 Recent Triage Activity

The right panel shows the most recent AI analysis sessions. Each row contains Bug ID, Severity, Confidence, a brief Root Cause, Duration, and a timestamp. Click the View button to open the full analysis result.

TIP: Dashboard numbers refresh automatically. If the data seems stale, refresh your browser tab.

6. Bug List

The Bug List is the main workspace of the application. It aggregates bugs from every connected source into one view with tools to filter, search, and start AI triage sessions. Click Bugs in the navigation bar, or click the asterisk logo.

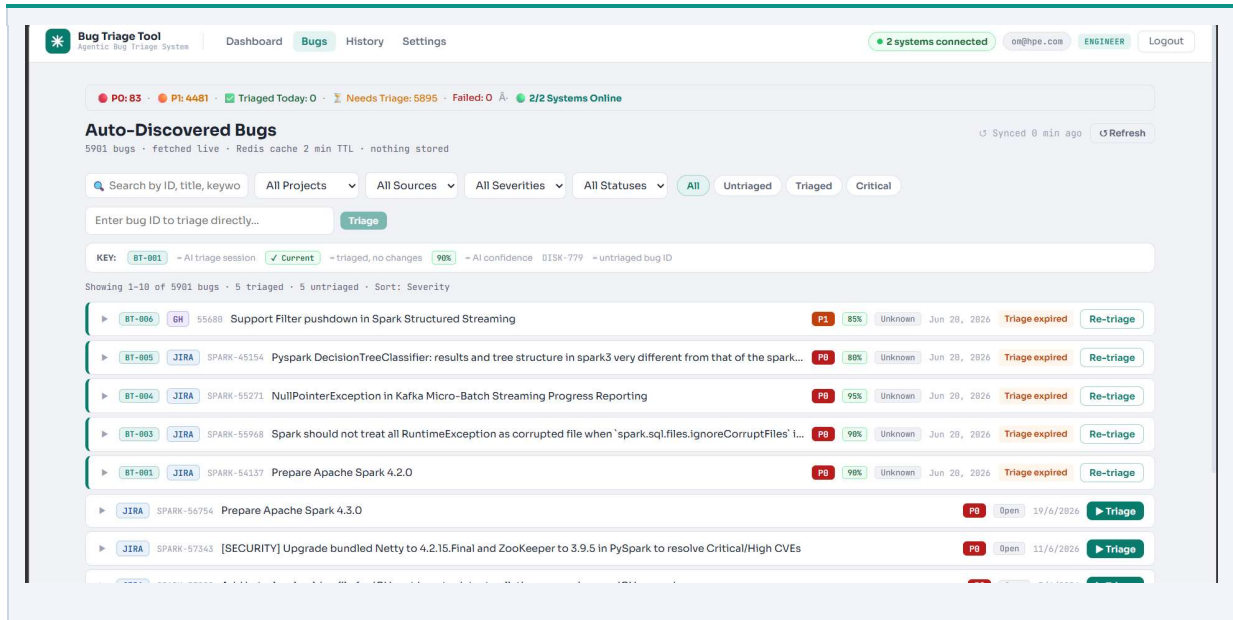


Figure 3 - Auto-Discovered Bugs: stats strip, filter/search bar, direct triage input, legend, and sorted bug rows

6.1 Mini Stats Strip

At the top of the Bug List there is a compact stats strip (the same counts as the Dashboard but smaller), letting you keep an eye on overall bug health without leaving the page.

6.2 Filtering and Searching

Use the filter bar to narrow down the bug list. You can combine any of these at once:

- Search box - type any word, phrase, or bug ID. The list updates as you type.
- All Projects - filter to a specific project key (e.g. KAFKA, SPARK).
- All Sources - show bugs from only one source system (GitHub, Jira, or Bugzilla).
- All Severities - show only bugs at a specific level (P0-P3).
- All Statuses - filter by source tracker status (Open, In Progress, Resolved, Closed).
- Pill buttons (All / Untriaged / Triaged / Critical) - one-click filter presets.

TIP: To clear all filters, click the "Clear filters" button that appears when no bugs match your current filter combination.

6.3 Direct Triage Bar

Below the filter bar is a text input that lets you triage any bug by its ID directly (e.g. "SPARK-57343"). Press Enter or click the Triage button to start the AI pipeline immediately without searching the list.

6.4 Understanding Bug Rows

Element	What It Shows
BT-001 badge (teal)	The internal triage session ID. Shown only on bugs that have been triaged.
GH / JIRA / BZ badge	The source system the bug came from (GitHub, Jira, or Bugzilla).
Ticket ID (e.g. SPARK-57343)	The original ticket ID from the source system, in monospace font.
Bug title	The title of the bug as it appears in the source tracker.
P0 / P1 / P2 / P3 badge	Severity: red = P0, orange = P1, yellow = P2, grey = P3.
Confidence % (e.g. 95%)	How confident the AI was in its severity classification.
Status pill	The current status in the source tracker (Open, In Progress, etc.).
Triage button	Starts a new AI analysis and takes you to the live Triage page.
Re-triage button	Runs a fresh triage on a previously analysed bug.

6.5 Triage State Indicators

State	Meaning	Action
Current	Bug has not changed since last triage. Analysis is valid.	View the previous result or run a fresh triage.
Triage Expired	Analysis data is older than 2 minutes and has been cleared from cache.	Click Re-triage to run the pipeline again.
No badge (untriaged)	This bug has never been through the AI pipeline.	Click the green Triage button to start analysis.

6.6 Pagination and Refresh

If more bugs exist than fit on one page, pagination controls appear at the bottom. The list auto-refreshes every two minutes. Click the Refresh button at the top right for an immediate reload.

7. Starting a Triage

Triaging a bug means sending it through the AI analysis pipeline. The pipeline gathers all available information, searches for related issues, looks up knowledge base articles, and produces a structured report with severity, root cause, and recommendations.

Option A - From the Bug List

1. Find the bug you want to triage in the Bug List
2. Click the green Triage button on the right side of the bug row
3. The system starts the AI pipeline and takes you to the Triage page automatically

Option B - By Bug ID

1. Find the Direct Triage bar below the filter bar on the Bug List page
2. Type the bug ID (e.g. SPARK-57343) into the input field
3. Press Enter or click the Triage button next to the input

IMPORTANT: While a triage is running, the button on that bug row shows "..." to indicate it is busy. Do not click it again - wait for the pipeline to finish.

8. Triage Page (Live Analysis)

The Triage page opens automatically when you start a triage. It shows the AI pipeline running in real time. Results appear as each stage completes - you do not have to wait for everything to finish before reading.

8.1 The Loading Screen

While the pipeline is running, you will see a loading card showing the four agents and their current state, a progress bar, and an elapsed time counter.

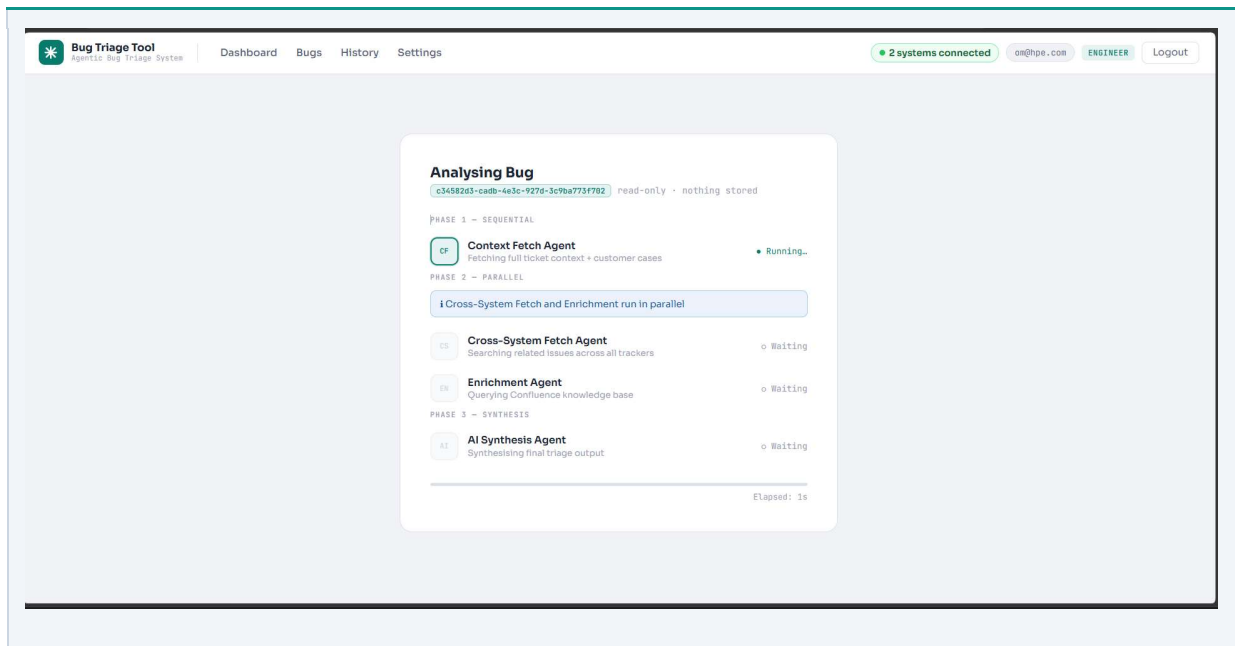


Figure 4 - Triage loading card: four agents (Phase 1-Sequential, Phase 2-Parallel, Phase 3-Sequential) with Running/Waiting status indicators

Agent	Phase	What It Does
Context Fetch Agent	Phase 1 (Sequential)	Retrieves the full ticket from the source system - title, description, all comments, assignee, linked items, customer signals, and change history.
Cross-System Fetch Agent	Phase 2 (Parallel)	Searches every other connected system for bugs similar or related to this one. Runs at the same time as the Enrichment Agent.

Agent	Phase	What It Does
Enrichment Agent	Phase 2 (Parallel)	Searches the connected knowledge base (Confluence or Support KB) for articles relevant to this bug.
AI Synthesis Agent	Phase 3 (Sequential)	Combines all gathered information and produces the final structured triage output: severity, confidence, root cause, and recommended actions.

NOTE: Phases 1 and 3 run one after the other. The two Phase 2 agents run in parallel to save time. The total pipeline typically takes 10-60 seconds.

8.2 The Results Grid

Once the first agent finishes, the page switches to a 2x2 results grid. You can read completed panels while others are still loading. Each panel has a coloured top border:

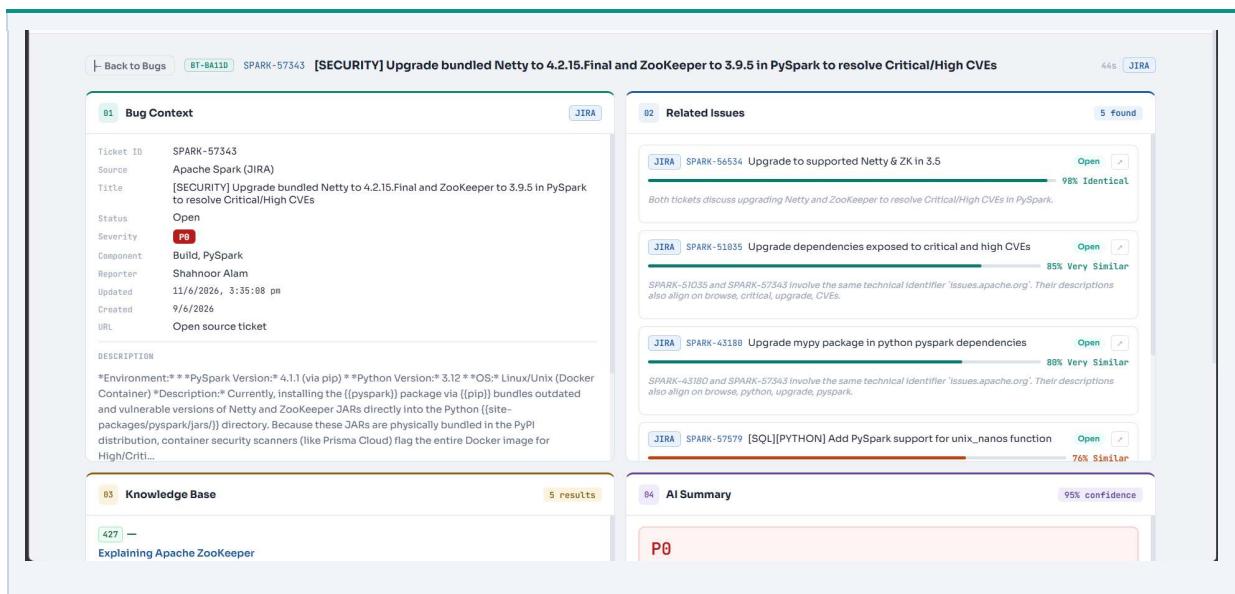


Figure 5 - Triage results grid: Bug Context (teal, top-left), Related Issues (blue, top-right), Knowledge Base (amber, bottom-left), AI Summary (purple, bottom-right)

Panel	Border Colour	Contents
01 - Bug Context	Teal	Full ticket details fetched from the source system.

Panel	Border Colour	Contents
02 - Related Issues	Blue	Similar bugs found in other connected systems.
03 - Knowledge Base	Amber	Relevant articles from Confluence or the Support KB.
04 - AI Summary	Purple	The final AI-generated triage result.

8.3 Panel 01 - Bug Context

This panel shows everything the system fetched from the original ticket: ticket ID, source, title, status, severity, component, assignee, reporter, dates, URL, full description, steps to reproduce, error excerpts, customer impact, recent comments, linked items, and public customer signals from sites such as Stack Overflow or Hacker News.

8.4 Panel 02 - Related Issues

This panel shows bugs from other connected systems that are similar to the bug being analysed. Each entry shows a source badge, ticket ID (click to open in the source system), a similarity bar with percentage score, and a brief explanation of why the AI considers these bugs related.

NOTE: Only bugs with a similarity score of 50% or higher are shown. If no bugs score above 50%, the panel will say "No related issues found".

8.5 Panel 03 - Knowledge Base

This panel shows articles from the connected knowledge base (usually Confluence) that are relevant to the bug. Each article shows a relevance score, a badge (High / Medium / Low), the article title as a clickable link, an excerpt, and associated tags.

8.6 Panel 04 - AI Summary

This is the most important panel. It shows the final structured triage output generated by the AI.

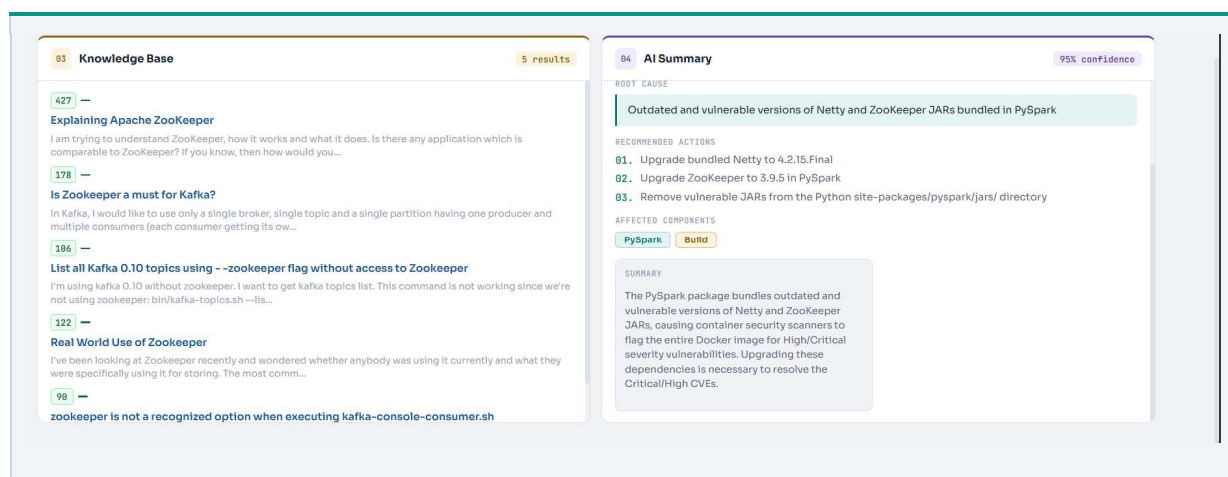


Figure 6 - Panel 03 Knowledge Base (left) and Panel 04 AI Summary (right): severity block, confidence bar, root cause, recommended actions, and affected components

Section	What It Shows
Severity block (P0/P1/P2/P3)	The AI severity classification in a large coloured box, with a rationale sentence.
Confidence bar	Percentage confidence in the classification. Green = high (80%+), amber = medium, red = low.
Root Cause	The AI's best hypothesis for why this bug is occurring.
Recommended Actions	A numbered list of concrete next steps for the engineering team.
Affected Teams	Colour-coded chips showing which teams should be involved.
Affected Components	Parts of the system the AI believes are involved in this bug.
Summary	A plain-English paragraph summarising the entire analysis.

IMPORTANT: If the panel shows "Fallback analysis used", the AI service was temporarily unavailable and a simpler rule-based analysis ran instead. Re-triage when the service is restored.

9. History Page

The History page shows a record of every completed triage session, storing up to the last 50. It is useful for tracking what has been analysed, reviewing past decisions, or re-running an analysis. Click History in the navigation bar.

TRiage ID	BUG ID	SOURCE	SEVERITY	CONFIDENCE	ROOT CAUSE	DURATION	TRIAGED AT	ACTIONS
BT-006	55680	GH	P1	85%	Lack of filter pushdown support in ...	20.6s	Jun 20, 2026 · 23:25	Triage expired Re-triage
BT-005	SPARK-45154	JIRA	P0	80%	Incompatible DecisionTreeClassif...	44.9s	Jun 20, 2026 · 23:19	Triage expired Re-triage
BT-004	SPARK-55271	JIRA	P0	95%	NullPointerException in KafkaMicr...	58.2s	Jun 20, 2026 · 23:19	Triage expired Re-triage
BT-003	SPARK-55968	JIRA	P0	90%	Incorrect handling of RuntimeExce...	1.1s	Jun 20, 2026 · 23:17	Triage expired Re-triage
BT-002	SPARK-55968	JIRA	P0	90%	Incorrect handling of RuntimeExce...	43.4s	Jun 20, 2026 · 23:16	Triage expired Re-triage
BT-001	SPARK-54137	JIRA	P0	90%	Insufficient preparation for Apach...	51.2s	Jun 20, 2026 · 23:13	Triage expired Re-triage

Figure 7 - Triage History: table with Triage ID, Bug ID, Source, Severity, Confidence, Root Cause, Duration, Triaged At, and Actions columns

9.1 Reading the History Table

Column	What It Shows
Triage ID	A unique internal ID for the session (e.g. BT-006). Every triage session gets one.
Bug ID	The ID of the analysed bug (e.g. SPARK-45154), in teal monospace font.
Source	A badge showing which system the bug came from (GH, JIRA, or BZ).
Severity	The AI-assigned severity level (P0, P1, P2, or P3).
Confidence	The AI confidence score as a percentage.
Root Cause	The first ~80 characters of the AI's root cause explanation.
Duration	How long the pipeline took to complete (e.g. "44.9s").

Column	What It Shows
Triaged At	The exact date and time the analysis finished.
Actions	Buttons to view the result or run a fresh triage.

9.2 Actions You Can Take

- View Results - opens the full saved output. Greyed out and replaced with "Triage expired" if the result is no longer in the live cache.
- Re-triage - runs the AI pipeline again. Useful when a bug has been updated or the previous result has expired.

10. Settings Page

The Settings page is where you configure the source systems that feed data into the Bug List. Click Settings in the navigation bar to open it.

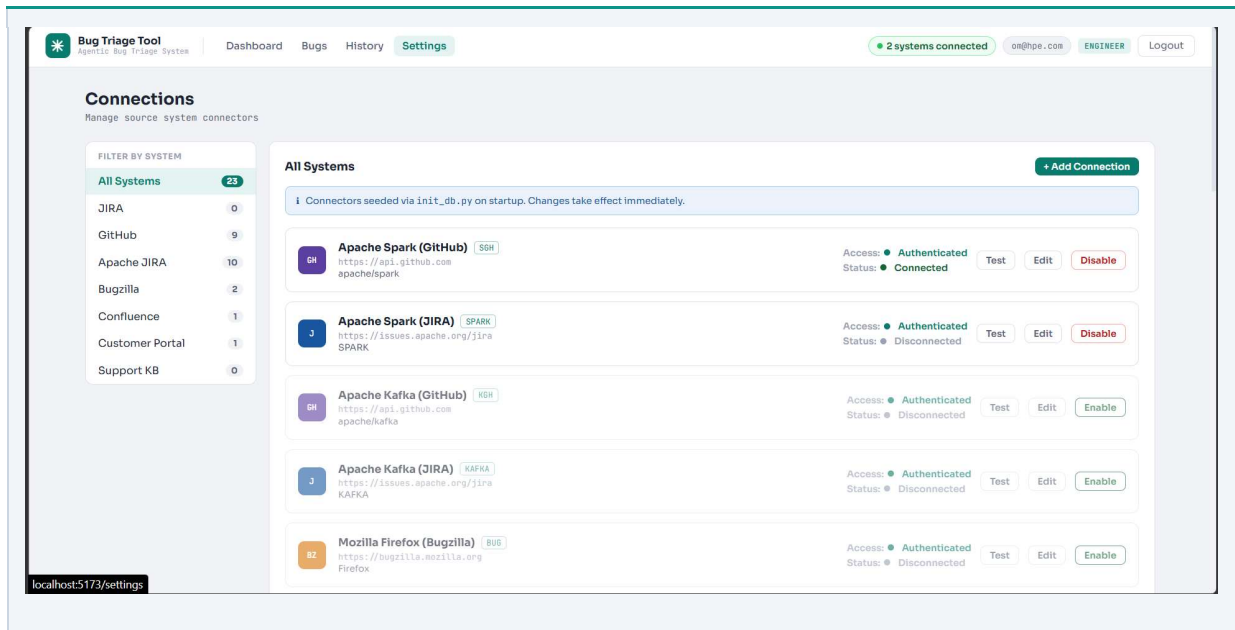


Figure 8 - Settings Connections tab: filter sidebar, connection cards with system name, Access/Status badges, and Test/Edit/Enable/Disable buttons

10.1 Connection Status Badges

Status	Colour	Meaning
Connected	Green	The last connection test succeeded. The system is reachable and providing data.
Disconnected	Grey	The connection is disabled or has never been tested.
Error	Red	The last test returned an error. Check the URL and credentials.
Timeout	Amber	The last test timed out. The system may be slow or the URL may be wrong.

10.2 Adding a New Connection

Click the + Add Connection button. A form will appear:

1 Enter a Display Name: A clear identifier, e.g. "Apache Kafka JIRA" or "Mozilla Bugzilla".

2 Choose the System Type: Select from: JIRA, GitHub Issues, Apache JIRA, Bugzilla, Confluence, Customer Portal, or Support KB.

3 Confirm or Update the Base URL: Check the pre-filled URL. Update it if you are connecting to a self-hosted instance.

4 Enter Your Authentication Token: Paste your personal access token, API key, or bearer token.

5 Enter a Project Key (if applicable): For JIRA and GitHub, enter the project or repository name (e.g. "KAFKA" or "apache/spark").

6 Click Save, then Test: Wait a few seconds, then click Test on the new connection card to confirm it is working.

TIP: After adding a connection, the Bug List will include bugs from that source within the next refresh cycle (up to 60 seconds). Refresh the Bug List page to see new bugs immediately.

10.3 Managing Existing Connections

- Edit - change any field (name, URL, token, project key). Click Save to apply changes.
- Test - verify the connection. A green Connected badge confirms success.
- Disable / Enable - temporarily toggle a connection without deleting it.
- Delete - permanently remove the connection. Bugs from that source will no longer appear after the next refresh.

IMPORTANT: Deleting a connection does not delete bugs from the source system. It only removes the link between this application and that system.

11. Adding a Custom Connector

If you need to connect a platform that is not built in (e.g. ServiceNow, Linear, or a proprietary tracker), you can write a new connector in five steps. Every connector is a Python class that extends `BaseConnector` and implements five required methods.

NOTE: You need access to the project source code and a running local development environment to complete these steps.

11.1 Overview

The connector system is designed so that adding a new platform requires changes in exactly two places: one new Python file and one line in `registry.py`. The Settings UI handles everything else automatically.

11.2 The Five Steps

1

Create a new Python class: Create a new file and write a class that extends `BaseConnector`. This means the new class inherits the base structure and must now fill in the platform-specific details.

```
# Example: connectors/servicenow_connector.py from base_connector import
BaseConnector class ServiceNowConnector(BaseConnector): pass #
implement the 5 methods below
```

2

Override the 5 required methods: `BaseConnector` has 5 abstract methods that every connector must implement:

Method	What the Engineer Writes
<code>get()</code>	Call the platform's API to fetch one ticket by ID.
<code>search()</code>	Call the platform's API to list open bugs.
<code>get_linked_items()</code>	Extract cross-references to other tickets.
<code>get_lightweight()</code>	Quick call to fetch only status and severity.

Method	What the Engineer Writes
<code>extract_links()</code>	Scan raw payload for mentions of other platform tickets.

3

Write the normalisation logic: Inside each method, map the platform's raw JSON fields to the system's standard TicketData fields.

```
# Map platform fields to TicketData standard fields: # title, description,
severity (P0-P3), # status, assignee, reporter, created_at, updated_at, #
comments, linked_items, url
```

The normalisation step is critical. The AI pipeline expects a consistent TicketData object regardless of the source platform. Map every field the platform provides; use None for fields the platform does not expose.

4

Register it in one line: In `registry.py`, add one entry to the `SYSTEM_TYPE_TO_CLASS` dictionary:

```
# registry.py SYSTEM_TYPE_TO_CLASS = {      "jira":          JiraConnector,
"github":      GitHubConnector,      "bugzilla":      BugzillaConnector,
"servicenow":  ServiceNowConnector,  # add this line }
```

That single line is all the backend needs. The system will now recognise "servicenow" as a valid system type when adding connections in the Settings UI.

5

Add it via the Settings UI: Go to Settings - Add Connection - select your new system type from the dropdown - enter the URL and authentication token - click Save. The system auto-tests the connection. If it passes, the connector is live instantly and bugs will begin appearing in the Bug List on the next refresh.

TIP: If the system type does not appear in the Settings dropdown, restart the backend service to reload `registry.py`. No other restart or rebuild is required.

11.3 Quick Reference - Connector Checklist

Step	File	What to Do
1	connectors/<name>_connector.py	Create the file and extend BaseConnector.
2	connectors/<name>_connector.py	Implement all 5 abstract methods.
3	connectors/<name>_connector.py	Map platform fields to TicketData inside each method.
4	registry.py	Add one line to SYSTEM_TYPE_TO_CLASS.
5	Settings UI	Add Connection - select type - enter URL and token - Save - Test.

12. Common Tasks - Quick Reference

This section summarises the most common things you will do in the application, with the fastest path to completing each one.

I want to...	How to do it
See all open bugs	Click Bugs in the navigation bar.
Find a specific bug by ID	Type the bug ID in the search box on the Bug List, or use the Direct Triage bar.
See only critical bugs	Click the Critical pill button on the Bug List page (filters to P0 and P1 bugs only).
See only untriaged bugs	Click the Untriaged pill button on the Bug List page.
Triage a bug	Find the bug in the Bug List and click the green Triage button on the right of its row.
Read the AI analysis for a bug	Click "View Previous Results" on the bug row, or go to History and click "View Results".
Re-run an analysis	Expand the bug row in the Bug List and click "Re-triage", or use the History page.
Check overall system health	Click Dashboard in the navigation bar.
See all past analyses	Click History in the navigation bar.
Add a new source system	Settings - Connections - Add Connection.
Check if a connection is working	Settings - Connections - click the Test button on the connection card.
Add a custom connector	See Section 11 - follow the 5-step process to create and register a new connector class.

13. Troubleshooting

This section covers the most common issues you may encounter and explains what to do about each one.

13.1 I Cannot Log In

- Double-check that your email and password are correct - passwords are case-sensitive
- Make sure there are no extra spaces before or after the email or password
- If you have forgotten your password, ask your project lead to reset it for you
- The message "Login failed. Check your credentials" means the combination does not match any account

13.2 The Bug List Is Empty

- Check that at least one connection is set up and enabled in Settings - Connections
- Click the Test button on each connection to confirm it shows "Connected"
- Click the Refresh button on the Bug List page to force a data reload
- If you just added a new connection, wait up to 60 seconds for the background sync to run
- If the page shows "Fetching live data... (first load)", wait a few seconds - the data is loading

13.3 The Triage Pipeline Is Stuck on "Running..."

- Wait at least 60 seconds - some pipelines take longer when source systems respond slowly
- Refresh the page - if the result completed, it will load from the stored cache
- If still stuck after 2 minutes, return to the Bug List and click Triage again
- Contact your project lead if the problem persists - the backend service may need restarting

13.4 The Page Shows "Triage Expired"

This means the detailed analysis data was cleared from the live cache (this happens after 2 minutes). It does not mean the analysis was wrong. Click the Re-triage button to run a fresh analysis. The previous severity and confidence score remain visible in the History page.

13.5 AI Summary Shows "Fallback Analysis Used"

The AI service was temporarily unavailable when the analysis ran. A simpler rule-based analysis was used instead. The severity rating may be less accurate. Click Re-triage to try again - the AI service is usually restored within a few minutes.

13.6 A Connection Shows "Error" or "Timeout"

- Click Edit on the connection card and verify the Base URL is correct
- Ensure the authentication token has not expired - generate a new one from the source system and update it here

- For GitHub connections, confirm the token has read access to the configured repository
- For Jira connections, verify the project key is correct (it is case-sensitive)
- Click Test again after making changes to confirm the connection is now working

13.7 My Custom Connector Type Does Not Appear in the Settings Dropdown

- Check that your class is correctly added to `SYSTEM_TYPE_TO_CLASS` in `registry.py`
- Restart the backend service to reload `registry.py`
- Check the backend logs for any import errors in your connector file
- Ensure your class correctly extends `BaseConnector` and all 5 methods are implemented

14. Glossary

Term	Definition
Agent	One of the four automated steps in the triage pipeline (Context Fetch, Cross-System Fetch, Enrichment, AI Synthesis).
BaseConnector	The abstract Python base class that every source connector must extend. Defines the 5 required methods.
BT-001	An internal triage session ID assigned by this system, starting with "BT-" followed by a number.
Confidence Score	A percentage (0-100%) showing how certain the AI is about its severity classification. Higher is more reliable.
Connection	A configured link between this application and a source system (GitHub, Jira, Bugzilla, or a knowledge base).
Connector	A Python class that extends BaseConnector and implements the 5 abstract methods for a specific platform.
Cross-System Fetch	The process of searching all connected systems for bugs similar to the one being triaged.
Enrichment	The process of finding relevant knowledge base articles to assist with bug investigation.
Knowledge Base	A documentation system (e.g. Confluence) containing articles relevant to known issues.
Normalisation	The process of mapping a platform's raw JSON fields to the system's standard TicketData fields.
P0 / P1 / P2 / P3	Severity levels: P0 = Critical, P1 = High, P2 = Medium, P3 = Low.
Panel	One of the four result sections on the Triage and Results pages. Each is produced by a different agent.
Pipeline	The full four-step AI analysis process that runs when you triage a bug.
Project Key	A short uppercase code identifying a project in Jira or a repository in GitHub (e.g. KAFKA, SPARK).

Term	Definition
registry.py	The file containing SYSTEM_TYPE_TO_CLASS - the dictionary that maps system type strings to connector classes.
Root Cause	The AI's hypothesis for why the bug is occurring, based on all gathered information.
Source Badge	A small coloured badge (GH, JIRA, BZ) showing which system a bug came from.
Source System	Any external bug tracker or knowledge base connected to this application.
TicketData	The standard data model all connectors must return. Contains fields like title, description, severity, assignee, etc.
Triage	The act of analysing a bug using the AI pipeline to determine severity, root cause, and recommended actions.
Triage Expired	Message shown when analysis data is no longer in the live cache. Re-triage to get fresh results.